



École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT) — Cloud et Sécurité
Stage réalisé au sein de Vermeg

Journal de Stage

Conception et développement d'une plateforme serverless de supervision,
d'audit et de détection des menaces liées aux modifications AWS IAM

Étudiant(e) :	Zouari Omar
Organisme d'accueil :	Vermeg
Encadrant professionnel :	Mr.Amine Chtioui
Période de stage :	1 ^{er} juillet 2026 – 31 août 2026
Année universitaire :	1 ^{re} année cycle ingénieur
Projet support :	<code>aws-iam-monitor</code>

Table des matières

Partie A — Calendrier du stage	2
Partie B — Journal de stage jour par jour	4

Partie A — Calendrier du stage

Le stage couvre la période du **1^{er} juillet 2026** au **31 août 2026**, soit 62 jours calendaires. Le calendrier des jours effectivement travaillés a été établi selon les règles suivantes : suppression de tous les samedis et dimanches, puis suppression des jours fériés tunisiens tombant un jour ouvré sur la période.

Élément	Valeur
Période du stage	01/07/2026 – 31/08/2026
Nombre de jours calendaires	62
Samedis et dimanches retirés	18
Jours fériés tombant un jour ouvré retirés	2
Nombre total de jours travaillés	42

Jours fériés tunisiens concernant la période

Date	Jour	Désignation
25/07/2026	Samedi	Fête de la République (tombe un week-end, sans impact sur le calendrier des jours travaillés)
13/08/2026	Jeudi	Fête de la Femme et de la Famille (jour ouvré retiré)
25/08/2026	Mardi	Al Mawlid, anniversaire du Prophète – <i>date fixée selon le calendrier lunaire, sous réserve de confirmation officielle</i> (jour ouvré retiré)

Organisation générale

Le stage est structuré en trois grandes phases, détaillées dans le journal ci-après :

- **Phase 1 – Découverte et apprentissage AWS** : 5 jours (01–07 juillet 2026) ;
- **Phase 2 – Développement du projet aws-iam-monitor** : 32 jours (08 juillet – 21 août 2026) ;
- **Phase 3 – Finalisation, documentation et préparation de la soutenance** : 5 jours (24–31 août 2026).

Calendrier détaillé des jours travaillés

Date	Jour	Statut
01/07/2026	Mercredi	Travaillé
02/07/2026	Jeudi	Travaillé
03/07/2026	Vendredi	Travaillé
04/07/2026	Samedi	Repos (week-end)
05/07/2026	Dimanche	Repos (week-end)
06/07/2026	Lundi	Travaillé
07/07/2026	Mardi	Travaillé
08/07/2026	Mercredi	Travaillé
09/07/2026	Jeudi	Travaillé
10/07/2026	Vendredi	Travaillé
11/07/2026	Samedi	Repos (week-end)
12/07/2026	Dimanche	Repos (week-end)
13/07/2026	Lundi	Travaillé

Date	Jour	Statut
14/07/2026	Mardi	Travaillé
15/07/2026	Mercredi	Travaillé
16/07/2026	Jeudi	Travaillé
17/07/2026	Vendredi	Travaillé
18/07/2026	Samedi	Repos (week-end)
19/07/2026	Dimanche	Repos (week-end)
20/07/2026	Lundi	Travaillé
21/07/2026	Mardi	Travaillé
22/07/2026	Mercredi	Travaillé
23/07/2026	Jeudi	Travaillé
24/07/2026	Vendredi	Travaillé
25/07/2026	Samedi	Repos (week-end) / Fête de la République
26/07/2026	Dimanche	Repos (week-end)
27/07/2026	Lundi	Travaillé
28/07/2026	Mardi	Travaillé
29/07/2026	Mercredi	Travaillé
30/07/2026	Jeudi	Travaillé
31/07/2026	Vendredi	Travaillé
01/08/2026	Samedi	Repos (week-end)
02/08/2026	Dimanche	Repos (week-end)
03/08/2026	Lundi	Travaillé
04/08/2026	Mardi	Travaillé
05/08/2026	Mercredi	Travaillé
06/08/2026	Jeudi	Travaillé
07/08/2026	Vendredi	Travaillé
08/08/2026	Samedi	Repos (week-end)
09/08/2026	Dimanche	Repos (week-end)
10/08/2026	Lundi	Travaillé
11/08/2026	Mardi	Travaillé
12/08/2026	Mercredi	Travaillé
13/08/2026	Jeudi	Jour férié – Fête de la Femme et de la Famille
14/08/2026	Vendredi	Travaillé
15/08/2026	Samedi	Repos (week-end)
16/08/2026	Dimanche	Repos (week-end)
17/08/2026	Lundi	Travaillé
18/08/2026	Mardi	Travaillé
19/08/2026	Mercredi	Travaillé
20/08/2026	Jeudi	Travaillé
21/08/2026	Vendredi	Travaillé
22/08/2026	Samedi	Repos (week-end)
23/08/2026	Dimanche	Repos (week-end)
24/08/2026	Lundi	Travaillé
25/08/2026	Mardi	Jour férié – Al Mawlid (sous réserve)
26/08/2026	Mercredi	Travaillé
27/08/2026	Jeudi	Travaillé
28/08/2026	Vendredi	Travaillé
29/08/2026	Samedi	Repos (week-end)
30/08/2026	Dimanche	Repos (week-end)
31/08/2026	Lundi	Travaillé

Partie B — Journal de stage jour par jour

Phase 1 — Découverte et apprentissage AWS (01 – 07 juillet 2026)

Mercredi 01/07/2026

Jour 1

Activités réalisées : Prise de contact avec l'organisme d'accueil, cadrage des objectifs du stage et du projet support (`aws-iam-monitor`). Mise en place de l'environnement de travail : installation et configuration d'AWS CLI, création d'un compte de travail restreint, premiers repères dans la console AWS IAM.

Objectif : Disposer d'un environnement fonctionnel avant d'aborder l'étude des services AWS et de sécuriser l'accès au compte de travail.

Technologies / outils : AWS CLI, console AWS, AWS IAM.

Résultats / acquis : Environnement de travail opérationnel ; premières notes prises dans le dépôt personnel de ressources AWS.

Jeudi 02/07/2026

Jour 2

Activités réalisées : Étude du service AWS IAM : utilisateurs, groupes, rôles, politiques (policies) managées et en ligne, mécanismes d'attachement des permissions et principe du moindre privilège.

Objectif : Acquérir les bases nécessaires à la compréhension du modèle de permissions AWS, préalable indispensable à un projet de supervision IAM.

Technologies / outils : AWS IAM, AWS CLI.

Résultats / acquis : Compréhension du modèle utilisateurs/groupes/rôles/politiques et des risques associés à une mauvaise configuration IAM.

Vendredi 03/07/2026

Jour 3

Activités réalisées : Étude d'AWS CloudTrail : notion de *management event*, structure d'un événement (acteur, action, ressource, adresse IP source), cas d'usage en audit et en détection de sécurité.

Objectif : Comprendre comment CloudTrail capture les appels d'API IAM, ce qui conditionne toute la chaîne de détection du futur projet.

Technologies / outils : AWS CloudTrail.

Résultats / acquis : Compréhension du format des événements CloudTrail et de leur exploitation pour la détection d'opérations sensibles.

Lundi 06/07/2026

Jour 4

Activités réalisées : Étude d'Amazon EventBridge (règles, *event pattern*, cibles) et d'AWS Lambda (modèle d'exécution serverless, runtimes disponibles, rôle d'exécution IAM).

Objectif : Comprendre le mécanisme de filtrage événementiel et le fonctionnement d'une fonction serverless, briques centrales de l'architecture à concevoir.

Technologies / outils : Amazon EventBridge, AWS Lambda, Python.

Résultats / acquis : Compréhension du couple EventBridge/Lambda comme mécanisme de traitement événementiel en quasi temps réel.

Mardi 07/07/2026

Jour 5

Activités réalisées : Étude d'Amazon SNS (topics, abonnements), Amazon S3 (chiffrement, versioning) et Amazon CloudWatch (logs, métriques personnalisées, alarmes). Synthèse de la semaine et cadrage définitif du sujet de projet : plateforme serverless de supervision et d'audit des changements IAM.

Objectif : Clore la phase de découverte AWS en couvrant l'ensemble des services identifiés comme nécessaires, puis formaliser le périmètre du projet à développer.

Technologies / outils : Amazon SNS, Amazon S3, Amazon CloudWatch.

Résultats / acquis : Vision d'ensemble des services AWS mobilisés par le projet ; sujet de stage formalisé autour du dépôt `aws-iam-monitor`.

Observations : Cette semaine a été consacrée exclusivement à l'apprentissage ; aucun développement n'a encore été entamé sur le projet.

Phase 2 — Développement du projet `aws-iam-monitor` (08 juillet – 21 août 2026)

Mercredi 08/07/2026

Jour 6

Activités réalisées : Analyse du besoin de sécurité : identification des opérations IAM sensibles à surveiller en priorité (`CreateUser`, `DeletePolicy`, `CreateAccessKey`, `DeactivateMFADevice`, `AttachUserPolicy`) et des scénarios de menace associés (création de compte non autorisée, désactivation de MFA, escalade de privilèges).

Objectif : Cadrer précisément ce que le système doit détecter avant de concevoir l'architecture technique.

Technologies / outils : AWS IAM, AWS CloudTrail.

Résultats / acquis : Liste des opérations IAM jugées critiques pour la supervision, base de la future logique de classification des risques.

Jeudi 09/07/2026

Jour 7

Activités réalisées : Étude approfondie de la manière dont CloudTrail journalise les appels d'API IAM au niveau compte, et réflexion sur le filtrage de ces événements par Amazon EventBridge (event pattern ciblant la source `iam.amazonaws.com`).

Objectif : Préparer la conception du flux de données qui reliera CloudTrail à la couche de traitement.

Technologies / outils : AWS CloudTrail, Amazon EventBridge.

Résultats / acquis : Principe de filtrage retenu : un *event pattern* EventBridge large sur la source IAM, le filtrage fin des actions surveillées étant délégué à la logique applicative de la Lambda.

Vendredi 10/07/2026

Jour 8

Activités réalisées : Conception du flux général de l'architecture : IAM → CloudTrail → EventBridge → Lambda → (SNS + S3 + CloudWatch). Choix motivé des services AWS pour chaque étage (capture, filtrage, traitement, notification, archivage, supervision).

Objectif : Fixer une architecture serverless cohérente avant tout développement.

Technologies / outils : AWS CloudTrail, Amazon EventBridge, AWS Lambda, Amazon

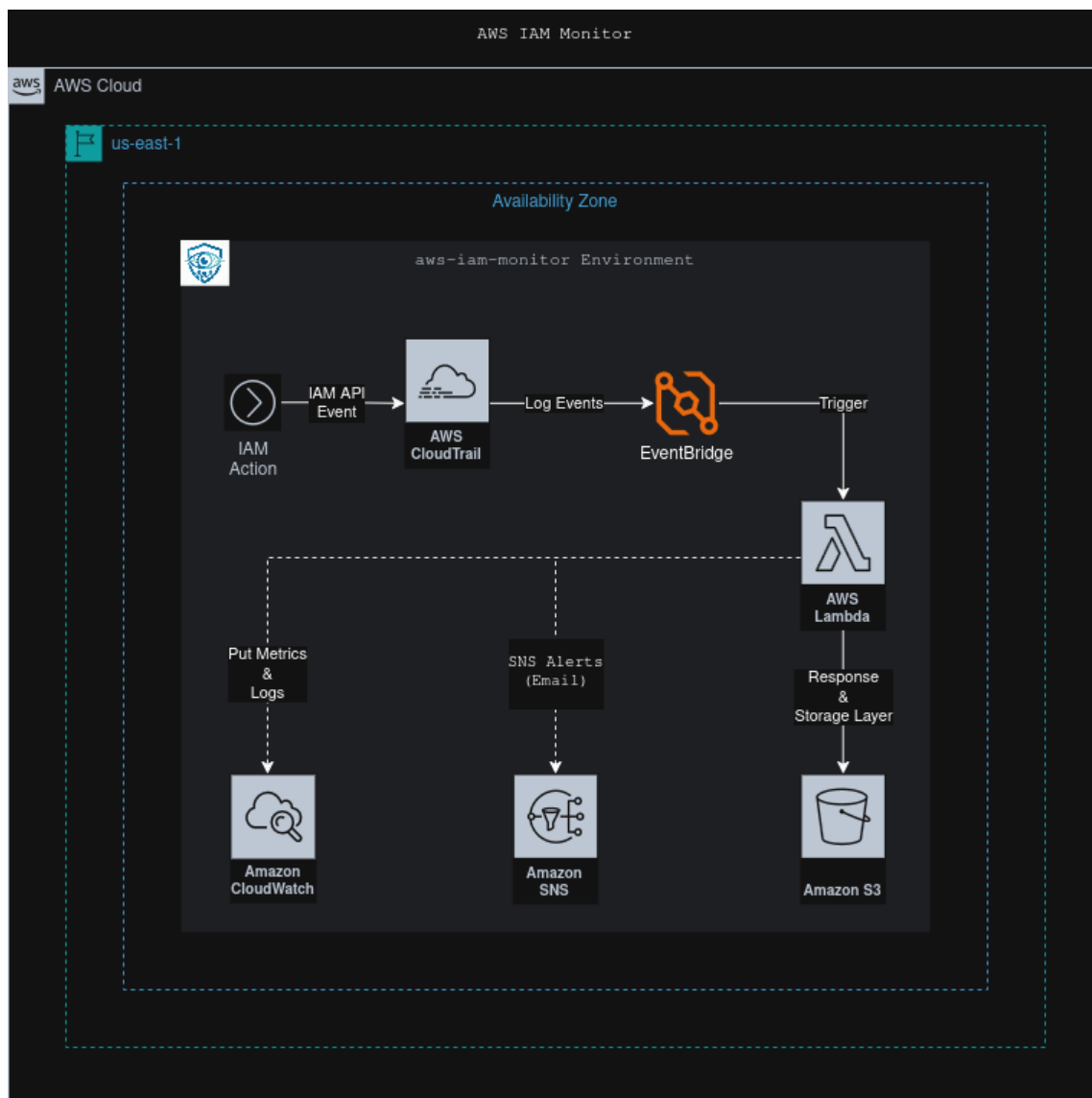


Figure 1: Architecture cible du projet `aws-iam-monitor`, retenue à l'issue de la phase de conception (Jour 8) et provisionnée à l'identique par les deux implémentations développées durant cette phase.

SNS, Amazon S3, Amazon CloudWatch.

Résultats / acquis : Schéma d'architecture validé, qui servira de référence pour les deux implémentations (Bash et Terraform).

Lundi 13/07/2026

Jour 9

Activités réalisées : Conception de la stratégie de classification des risques (niveaux CRITICAL, HIGH, MEDIUM, LOW) et de la logique de *whitelist* d'adresses IP, destinée à qualifier une action IAM comme provenant ou non d'une source de confiance.

Objectif : Définir les règles métier de la fonction Lambda avant son implémentation.

Technologies / outils : AWS Lambda (conception), AWS IAM.

Résultats / acquis : Grille de classification des risques et principe de génération dynamique d'une politique IPWhitelistPolicy.

Mardi 14/07/2026**Jour 10**

Activités réalisées : Démarrage de l'implémentation Bash/AWS CLI : mise en place de la structure du répertoire `project/bash/` (bibliothèque partagée `lib/` avec les fonctions communes, le module de journalisation et la vérification des prérequis) et squelette du script d'orchestration `awsctl`.

Objectif : Poser une base modulaire et réutilisable avant d'implémenter chaque composant AWS.

Technologies / outils : Bash, AWS CLI.

Résultats / acquis : Squelette fonctionnel de `awsctl` et bibliothèque d'aide commune aux différents modules.

Mercredi 15/07/2026**Jour 11**

Activités réalisées : Développement du module IAM (Bash) : création des politiques gérées `DevPolicy`, `ProdPolicy` et `AuditPolicy`, ainsi que des groupes `DevGroup`, `ProdGroup` et `TestGroup`.

Objectif : Mettre en place le bac à sable IAM de démonstration nécessaire au fonctionnement du système.

Technologies / outils : AWS IAM, AWS CLI, Bash.

Résultats / acquis : Politiques et groupes IAM créés de façon scriptée et idempotente.

Jeudi 16/07/2026**Jour 12**

Activités réalisées : Poursuite du module IAM (Bash) : création des utilisateurs de démonstration (`DevAdmin`, `ProdAdmin`, etc.), ajout de l'option `-r/-role` pour la création optionnelle de rôles IAM, et génération dynamique de la politique `IPWhitelistPolicy` à partir de la variable `ALLOWED_IPS`.

Objectif : Compléter le module IAM avec la gestion des utilisateurs et la logique de restriction par adresse IP.

Technologies / outils : AWS IAM, Bash.

Résultats / acquis : Module IAM complet, incluant la génération conditionnelle de rôles et de la politique de whitelist.

Vendredi 17/07/2026**Jour 13**

Activités réalisées : Développement du module CloudTrail (Bash) : création du bucket S3 destiné aux journaux (chiffrement AES-256, versioning activé) puis du *trail* CloudTrail capturant les événements de management sur l'ensemble du compte.

Objectif : Mettre en place la source d'événements sur laquelle repose toute la chaîne de détection.

Technologies / outils : AWS CloudTrail, Amazon S3, Bash.

Résultats / acquis : Trail CloudTrail opérationnel, journalisant les appels d'API dans un bucket S3 sécurisé.

Lundi 20/07/2026**Jour 14**

Activités réalisées : Développement du module SNS (Bash) : création du topic d'alerte `iam-alerts`, définition de sa politique d'accès, et abonnement automatique d'une adresse e-mail si la variable `SNS_ALERT_EMAIL` est renseignée dans le fichier `.env`.

Objectif : Mettre en place le canal de notification des événements à haut risque.

Technologies / outils : Amazon SNS, Bash.

Résultats / acquis : Topic SNS fonctionnel avec abonnement e-mail configurable.

Mardi 21/07/2026**Jour 15**

Activités réalisées : Démarrage du module Lambda (Bash), première partie : création du bucket S3 d'historique d'audit, du rôle d'exécution de la fonction, et packaging initial du code `lambda_function.py` (Python 3.11).

Objectif : Préparer l'infrastructure d'exécution avant d'implémenter la logique métier de traitement des événements.

Technologies / outils : AWS Lambda, Python 3.11, Amazon S3, AWS IAM, Bash.

Résultats / acquis : Rôle d'exécution et bucket d'audit provisionnés ; premier déploiement du squelette de la fonction Lambda.

Mercredi 22/07/2026**Jour 16**

Activités réalisées : Implémentation de la logique de traitement du payload CloudTrail reçu par la fonction Lambda, et de la vérification de l'adresse IP source de l'appel d'API par rapport à la politique de whitelist définie précédemment.

Objectif : Permettre à la fonction de distinguer une opération IAM légitime d'une opération provenant d'une source non autorisée.

Technologies / outils : Python, AWS Lambda.

Résultats / acquis : Fonction capable d'extraire les champs pertinents de l'événement et de qualifier la source de l'appel.

Jeudi 23/07/2026**Jour 17**

Activités réalisées : Implémentation de la classification du niveau de risque de chaque événement IAM et de l'archivage structuré des enregistrements JSON dans le bucket S3 d'audit, organisés par date (YYYY/MM/DD/).

Objectif : Assurer la traçabilité complète des événements IAM traités, quel que soit leur niveau de risque.

Technologies / outils : Python, Amazon S3, AWS Lambda.

Résultats / acquis : Chaque événement traité est classé et archivé de façon structurée et exploitable a posteriori.

Vendredi 24/07/2026**Jour 18**

Activités réalisées : Implémentation de l'émission de métriques personnalisées CloudWatch (namespace `AWSIAMMonitor`) et de la publication d'alertes SNS pour les événements classés à risque élevé, achevant ainsi la logique métier de la fonction Lambda.

Objectif : Compléter la fonction de traitement afin qu'elle notifie et alimente la supervision,

en plus de journaliser.

Technologies / outils : Amazon CloudWatch, Amazon SNS, Python.

Résultats / acquis : Fonction Lambda fonctionnellement complète : analyse, classification, archivage, alerte et métriques.

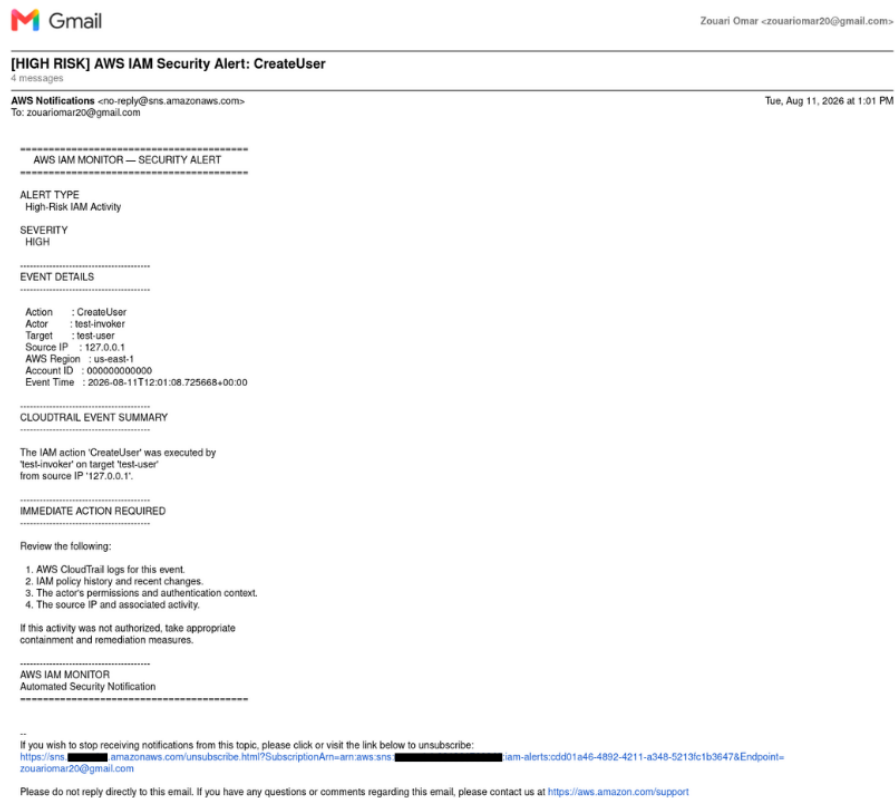


Figure 2: Exemple d'alerte SNS reçue par e-mail pour un événement IAM classé à risque élevé (CreateUser).

Lundi 27/07/2026

Jour 19

Activités réalisées : Développement du module EventBridge (Bash) : création de la règle de filtrage des événements IAM, rattachement de la fonction Lambda comme cible, et attribution de la permission d'invocation nécessaire.

Objectif : Fermer la boucle événementielle en connectant CloudTrail à la fonction Lambda via EventBridge.

Technologies / outils : Amazon EventBridge, AWS Lambda, Bash.

Résultats / acquis : Chaîne complète CloudTrail → EventBridge → Lambda opérationnelle de bout en bout.

Mardi 28/07/2026

Jour 20

Activités réalisées : Finalisation de l'orchestration `awsctl` : implémentation des actions `-up/-down/-status/-test`, des cibles (`all`, `iam`, `cloud-trail`, `sns`, `lambda`, `event-bridge`) et des options (`-r`, `-v`, `-h`).

Objectif : Offrir une interface en ligne de commande unique pour piloter l'ensemble de l'infrastructure Bash.

Technologies / outils : Bash, AWS CLI.

Résultats / acquis : `awsctl` permet de provisionner, auditer et détruire chaque composant individuellement ou l'ensemble de l'infrastructure.

Mercredi 29/07/2026

Jour 21

Activités réalisées : Mise en place de l'ordre de déploiement en 5 étapes (IAM → CloudTrail → SNS → Lambda → EventBridge) et gestion de la propagation IAM lors de la création de la fonction Lambda par une boucle de nouvelles tentatives avec temporisation.

Objectif : Fiabiliser l'enchaînement des créations de ressources malgré la latence de propagation IAM côté AWS.

Technologies / outils : Bash, AWS CLI, AWS IAM.

Résultats / acquis : Déploiement séquencé fiable, y compris lorsque le rôle IAM n'est pas immédiatement assumable par Lambda.

Jeudi 30/07/2026

Jour 22

Activités réalisées : Revue et amélioration de l'idempotence des scripts Bash : vérification systématique de l'existence d'une ressource avant création, et gestion propre des cas de mise à jour plutôt que de duplication.

Objectif : Permettre de relancer `awsctl -up` sans provoquer d'erreurs ni de doublons de ressources.

Technologies / outils : Bash, AWS CLI.

Résultats / acquis : Scripts robustes à une exécution répétée, comportement attendu d'une infrastructure gérée de façon impérative.

Vendredi 31/07/2026

Jour 23

Activités réalisées : Tests de bout en bout de l'implémentation Bash : `./awsctl -up all` pour le déploiement complet, `./awsctl -status all` pour la vérification, puis `./awsctl -down all` pour la désinstallation propre de l'ensemble des ressources.

Objectif : Valider que l'implémentation Bash provisionne et détruit correctement l'intégralité de l'architecture.

Technologies / outils : Bash, AWS CLI.

Résultats / acquis : Cycle complet provisioning/vérification/destruction validé sur l'implémentation Bash.

Observations : Cette validation clôture la première implémentation ; la suite du stage est consacrée à l'implémentation Terraform équivalente.

Lundi 03/08/2026

Jour 24

Activités réalisées : Démarrage de l'implémentation Terraform : mise en place de la structure du répertoire `project/terraform/` (`main.tf`, `variables.tf`, `outputs.tf`, `providers.tf`, `versions.tf`, `locals.tf`).

Objectif : Poser les fondations d'une implémentation déclarative équivalente à l'implémentation Bash.

Technologies / outils : Terraform, AWS Provider.

Résultats / acquis : Squelette du projet Terraform mis en place, prêt à accueillir les modules.

Mardi 04/08/2026**Jour 25**

Activités réalisées : Développement du module Terraform `s3` : bucket générique et réutilisable gérant chiffrement, versioning et blocage de l'accès public, instancié deux fois (bucket de logs CloudTrail et bucket d'historique d'audit).

Objectif : Factoriser la logique de création de bucket S3, commune aux deux usages du projet.

Technologies / outils : Terraform, Amazon S3.

Résultats / acquis : Module `s3` générique, sans politique de bucket propre (déléguée à l'appelant), prêt à être réutilisé.

Mercredi 05/08/2026**Jour 26**

Activités réalisées : Développement du module Terraform `cloudtrail` : création du trail et de la politique de bucket S3 spécifique à CloudTrail, appliquée sur l'instance du module `s3` dédiée aux journaux.

Objectif : Reproduire en Terraform le comportement du module CloudTrail Bash.

Technologies / outils : Terraform, AWS CloudTrail.

Résultats / acquis : Module `cloudtrail` fonctionnel, cohérent avec l'implémentation Bash.

Jeudi 06/08/2026**Jour 27**

Activités réalisées : Développement du module Terraform `iam` : bac à sable IAM de démonstration (politiques, groupes, utilisateurs, rôles optionnels) et génération dynamique de la politique de whitelist IP, module autonome sans dépendance croisée.

Objectif : Porter la logique IAM du projet vers une approche entièrement déclarative.

Technologies / outils : Terraform, AWS IAM.

Résultats / acquis : Module `iam` Terraform équivalent fonctionnellement au module Bash correspondant.

Vendredi 07/08/2026**Jour 28**

Activités réalisées : Développement du module Terraform `sns` : topic d'alerte de sécurité, politique du topic et abonnement e-mail optionnel, piloté par les variables Terraform.

Objectif : Reproduire le canal de notification en configuration entièrement pilotée par variables (`terraform.tfvars`).

Technologies / outils : Terraform, Amazon SNS.

Résultats / acquis : Module `sns` Terraform opérationnel, sans dépendance au fichier `.env` de l'implémentation Bash.

Lundi 10/08/2026**Jour 29**

Activités réalisées : Développement du module Terraform `lambda` : fonction de supervision, rôle et politique d'exécution, packaging du code source partagé avec l'implémentation Bash, redéploiement automatique piloté par `source_code_hash`.

Objectif : Éviter la duplication du code Python en réutilisant la même source Lambda pour les deux implémentations, tout en automatisant son déploiement.

Technologies / outils : Terraform, AWS Lambda, Python.

Résultats / acquis : Détection automatique des changements de code Lambda et redé-

ploiement déclaratif sans étape de packaging manuelle.

Mardi 11/08/2026

Jour 30

Activités réalisées : Développement du module Terraform `eventbridge` (règle de filtrage, cible Lambda, permission d'invocation) et intégration de l'ensemble des modules dans le fichier `main.tf` racine.

Objectif : Assembler les modules développés jusqu'ici en une architecture complète et cohérente.

Technologies / outils : Terraform, Amazon EventBridge.

Résultats / acquis : Architecture Terraform assemblée, l'ordre de déploiement étant résolu automatiquement par le graphe de dépendances des modules.

Mercredi 12/08/2026

Jour 31

Activités réalisées : Développement du module Terraform `cloudwatch` : groupe de logs de la fonction Lambda et alarmes optionnelles (`UnauthorizedIPAccess`, erreurs de la fonction Lambda), publiant vers le topic SNS existant.

Objectif : Ajouter une amélioration propre à l'implémentation Terraform, absente de l'implémentation Bash : l'exploitation active des métriques via des alarmes.

Technologies / outils : Terraform, Amazon CloudWatch, Amazon SNS.

Résultats / acquis : Alarmes CloudWatch optionnelles (activées par défaut via la variable `enable_cloudwatch_alarms`) déclenchant une notification SNS.

Vendredi 14/08/2026

Jour 32

Activités réalisées : Exécution complète du cycle `terraform init / plan / apply` sur l'ensemble des modules, puis vérification de l'équivalence fonctionnelle de l'architecture obtenue avec celle provisionnée par l'implémentation Bash.

Objectif : Valider que l'implémentation Terraform provisionne la même architecture cible que l'implémentation Bash.

Technologies / outils : Terraform, AWS CLI.

Résultats / acquis : Déploiement Terraform validé de bout en bout, avec les mêmes noms de ressources par défaut que l'implémentation Bash.

Lundi 17/08/2026

Jour 33

Activités réalisées : Comparaison détaillée des deux implémentations : portée plus stricte des permissions IAM de la Lambda côté Terraform, blocage explicite de l'accès public S3, et remplacement de la boucle de nouvelles tentatives Bash par une temporisation déterministe (`time_sleep`).

Objectif : Documenter précisément les différences volontaires entre les deux approches, au-delà de leur équivalence fonctionnelle globale.

Technologies / outils : Terraform, AWS IAM, Amazon S3.

Résultats / acquis : Tableau de comparaison technique Bash / Terraform consolidé.

Mardi 18/08/2026

Jour 34

Activités réalisées : Réflexion sur la sécurisation de l'état Terraform : limites du *backend* local (`terraform.tfstate`) pour un usage en équipe, et étude d'un *backend* distant S3 avec verrouillage DynamoDB pour un déploiement partagé.

Objectif : Anticiper les besoins d'un usage collaboratif de l'infrastructure Terraform, au-delà du cadre expérimental du stage.

Technologies / outils : Terraform, Amazon S3, Amazon DynamoDB (étude).

Résultats / acquis : Recommandations documentées sur la sécurité de l'état Terraform et la configuration d'un backend distant.

Mercredi 19/08/2026

Jour 35

Activités réalisées : Rédaction de la documentation technique du projet : `ARCHITECTURE.md` (schéma et tableau des services), `INSTALL.md` (prérequis des deux implémentations) et `USAGE.md` (référence complète des commandes `awsctl` et Terraform).

Objectif : Rendre le projet exploitable et compréhensible par une tierce personne sans connaissance préalable.

Technologies / outils : Markdown, Git.

Résultats / acquis : Documentation technique du dépôt consolidée et à jour avec l'état final du code.

Jeudi 20/08/2026

Jour 36

Activités réalisées : Mise en place et génération du site de documentation Sphinx (`docapi`), avec build local via `make docs` et prévisualisation du rendu HTML.

Objectif : Offrir une documentation navigable et centralisée du projet, complémentaire aux fichiers Markdown du dépôt.

Technologies / outils : Sphinx, Make.

Résultats / acquis : Site de documentation généré localement, structurant l'ensemble des pages Markdown existantes.

Vendredi 21/08/2026

Jour 37

Activités réalisées : Revue globale de l'architecture et du dépôt : relecture croisée des modules Bash et Terraform, nettoyage des fichiers de configuration d'exemple (`.env.example`, `terraform.tfvars.example`) et mise à jour du `CHANGELOG.md`.

Objectif : Clôturer la phase de développement sur une base de code propre et cohérente avant d'entamer la rédaction des livrables finaux.

Technologies / outils : Git, Bash, Terraform.

Résultats / acquis : Dépôt `aws-iam-monitor` dans un état stable, documenté et considéré comme fonctionnellement complet pour les deux implémentations.

Observations : Cette journée marque la fin de la phase de développement technique ; la dernière semaine du stage sera consacrée à la documentation et à la préparation de la soutenance.

Phase 3 — Finalisation, documentation et soutenance (24 – 31 août 2026)

Lundi 24/08/2026**Jour 38**

Activités réalisées : Reconstruction et rédaction du journal de stage jour par jour, à partir de l'historique réel des travaux menés sur le dépôt `aws-iam-monitor` et des notes d'apprentissage AWS.

Objectif : Disposer d'une trace fidèle et détaillée du déroulement du stage pour la soutenance et l'évaluation.

Technologies / outils : Traitement de texte, Git (historique du dépôt).

Résultats / acquis : Premier jet complet du journal de stage.

Mercredi 26/08/2026**Jour 39**

Activités réalisées : Rédaction des premiers chapitres du rapport de stage : contexte général, étude préalable du problème de sécurité IAM, et chapitre d'étude et de conception de la solution.

Objectif : Formaliser par écrit la démarche d'analyse et de conception ayant précédé le développement.

Technologies / outils : Traitement de texte.

Résultats / acquis : Chapitres 1 et 2 du rapport rédigés.

Jeudi 27/08/2026**Jour 40**

Activités réalisées : Rédaction des chapitres du rapport consacrés à la réalisation technique (par composant : IAM, CloudTrail, EventBridge, Lambda, SNS, S3, CloudWatch, Bash, Terraform) ainsi qu'au déploiement, à la validation et aux résultats.

Objectif : Détailler par écrit le travail de développement réellement effectué, en cohérence avec le journal de stage.

Technologies / outils : Traitement de texte.

Résultats / acquis : Chapitres 3 et 4 du rapport rédigés.

Vendredi 28/08/2026**Jour 41**

Activités réalisées : Rédaction du bilan des compétences acquises, des perspectives d'amélioration du projet et de la conclusion générale du rapport ; relecture et mise en forme de l'ensemble du document.

Objectif : Clore la rédaction du rapport par une synthèse fidèle du travail accompli et de ses limites.

Technologies / outils : Traitement de texte.

Résultats / acquis : Chapitre 5 et conclusion générale rédigés ; rapport de stage complet.

Lundi 31/08/2026**Jour 42**

Activités réalisées : Préparation de la soutenance : structuration du support de présentation, sélection des points techniques clés à démontrer (architecture, comparaison Bash/Terraform, classification des risques), et répétition des explications.

Objectif : Se présenter à la soutenance avec un support clair et une explication maîtrisée du projet.

Technologies / outils : Outil de présentation.

Résultats / acquis : Support de présentation finalisé, dernier jour du stage.

Observations : Fin du stage.